TUTORIAL DE GIT · CAPÍTULO 5

Git en entornos distribuidos

Flujos de trabajo colaborativos y cómo contribuir y mantener un proyecto

1. Flujos de trabajo distribuidos

Flujo	Descripción
Centralizado	Un único repositorio central; todos los desarrolladores sincronizan con él (hacen `pull` antes de `push`).
Gestor de integración	Cada desarrollador tiene su repositorio público; un gestor integra sus cambios en el repositorio "oficial" (el modelo de los *fork* y *pull request*).
Dictador y tenientes	Variante para proyectos enormes: los tenientes gestionan partes del proyecto y el dictador (por ejemplo, Linus Torvalds) integra el trabajo de los tenientes.

2. Contribuir a un proyecto

Pautas para los commits

- Revisar los errores de espacios en blanco antes de confirmar con `git diff --check`.
- Hacer **commits atómicos**: cada commit agrupa un cambio lógico y se entiende por sí solo.
- Escribir buenos mensajes: una línea de resumen de unos 50 caracteres como máximo, una línea en blanco y luego una explicación más detallada de qué y por qué.

```
Agrega función para calcular el promedio

Se incluye la función promedio() en estadistica.py porque el
módulo de reportes la necesita. Se agregan pruebas básicas.
```

Equipos pequeños y privados

Cada integrante hace `git pull` para traer lo último, trabaja en una rama puntual, la fusiona y hace `git push`. Es importante sincronizar con frecuencia para evitar conflictos grandes.

Proyectos públicos

- **Fork y pull request:** se hace una bifurcación (*fork*) del proyecto, se trabaja en una rama de tema y se solicita al mantenedor que integre los cambios mediante una solicitud de extracción (*pull request*).
- **Por correo electrónico:** se generan parches con `git format-patch` y se envían con `git send-email`.

3. Mantener un proyecto

```
git apply parche.diff        # aplica un parche al directorio de trabajo
git am parche.patch          # aplica un parche recibido por correo y crea el
commit
git cherry-pick <hash>       # trae un solo commit de otra rama
git remote add colaborador https://...   # agregar el repositorio de un
colaborador
```

```
git fetch colaborador
```

Los mantenedores suelen trabajar con **ramas de tema** para cada contribución: las prueban en una rama temporal y, si son correctas, las fusionan con la rama principal.

Rebase y cherry-pick

Antes de integrar una contribución, se puede pedir al autor que rebase su trabajo sobre la rama principal actual, para que la fusión sea limpia. `git cherry-pick` permite aplicar solo algunos commits específicos.

4. Publicar versiones

```
git describe main             # genera un nombre legible como v1.0-3-g1a2b3c4
git archive main --format=zip -o proyecto.zip   # crea un archivo comprimido
de la versión
git shortlog --no-merges main --not v1.0         # resumen de cambios desde
una versión
```

> *Resumen: Git permite varios flujos de colaboración; el más habitual en la actualidad es el de fork y pull request. Los commits pequeños, atómicos y con buenos mensajes facilitan el trabajo de todos.*

Fuente: Adaptado de Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2 (licencia CC BY-NC-SA 3.0).